1. What is Python?

Python is a **high-level, interpreted, general-purpose programming language** created by **Guido van Rossum** and first released in **1991**. It emphasises code readability and simplicity, using indentation to define code blocks instead of curly braces.

Feature	Description
Type	High-level, Interpreted
Creator	Guido van Rossum
First Released	1991
Current Version	Python 3.x
Typing	Dynamically Typed
Paradigm	OOP, Functional, Procedural
File Extension	.py

Key Features of Python

- **Easy to Learn:** Simple English-like syntax makes it beginner-friendly.
- **Interpreted:** Code runs line by line — no compilation needed.
- **Cross-Platform:** Works on Windows, macOS, and Linux without changes.
- **Open Source:** Free to use, distribute, and modify.
- **Large Standard Library:** Comes with hundreds of built-in modules.
- **Dynamically Typed:** No need to declare variable types explicitly.
- **Object-Oriented:** Supports classes, inheritance, and encapsulation.
- **Extensible:** Can integrate C, C++, and Java code.

■ Python was named after the British comedy group 'Monty Python', not the snake!

Pythonnad

2. Applications of Python

Python is one of the most versatile languages in the world. Below are the key domains where Python is heavily used:

Domain	Examples / Libraries
Web Development	Django, Flask, FastAPI
Data Science	Pandas, NumPy, Matplotlib
Machine Learning / AI	TensorFlow, PyTorch, Scikit-learn
Automation / Scripting	Selenium, PyAutoGUI
Cybersecurity	Scapy, Nmap, Paramiko
Game Development	Pygame, Panda3D
Desktop Applications	Tkinter, PyQt, Kivy
IoT / Embedded	MicroPython, Raspberry Pi

3. Installing Python

Step-by-step Installation Guide:

- Go to the official website: **www.python.org**
- Click on **"Downloads"** and choose your OS (Windows / macOS / Linux).
- Download the latest **Python 3.x** installer.
- Run the installer. On Windows, **check the box** that says *'Add Python to PATH'*.
- Click **Install Now** and wait for installation to complete.
- Open Command Prompt / Terminal and verify:

■ Verify Installation

```
python --version

# Expected output:
```

```
Python 3.12.x
```

■ Always install Python 3.x (not 2.x). Python 2 is no longer supported since Jan 1, 2020.

4. Python IDEs and Editors

An **IDE (Integrated Development Environment)** makes coding easier with features like auto-complete, syntax highlighting, and debugging tools.

IDE / Editor	Best For	Cost
IDLE	Beginners	Free (built-in)
VS Code	All Levels	Free
PyCharm	Professional	Free / Paid
Jupyter Notebook	Data Science	Free
Spyder	Scientific Python	Free
Thonny	Beginners	Free
Google Colab	ML / Cloud	Free

5. Your First Python Program

Every programmer's journey begins with the classic **"Hello, World!"** program. In Python, this is just one line:

■ hello.py — Your First Program

```
print("Hello, World!")
```

Output:

■ Output

```
Hello, World!
```

Breaking It Down:

- **print()** — A built-in function that displays output on the screen.
- **"Hello, World!"** — A *string* (text) passed as an argument inside quotes.
- Python does NOT require a semicolon **;** at the end of statements.
- Python is **case-sensitive**: print ≠ Print ≠ PRINT.

6. Variables and Data Types

A **variable** is a named container that stores a value in memory. In Python, you do **not need to declare the type** — Python infers it automatically.

Variable Declaration Syntax:

■ Variable Examples

```
variable_name = value


# Examples:

name = "Ali" # String

age = 20 # Integer

gpa = 3.75 # Float

is_pass = True # Boolean
```

Rules for Naming Variables:

- Must start with a **letter** or **underscore (_)**.
- Cannot start with a number.
- Can contain letters, numbers, and underscores.
- Cannot be a Python **keyword** (e.g., for, if, while, class).
- Are **case-sensitive**: age ≠ Age ≠ AGE.

Valid vs Invalid Variable Names:

■ Valid vs Invalid

```
# VALID

my_name = "Sara"

_count = 10

total2 = 500


# INVALID (will cause SyntaxError)

2total = 500 # starts with number

my-name = "Sara" # hyphen not allowed

for = 10 # 'for' is a keyword
```

7. Python Data Types

Python has several built-in data types. The **type()** function reveals the type of any variable.

Data Type	Keyword	Example	Description
Integer	int	x = 10	Whole numbers
Float	float	x = 3.14	Decimal numbers
String	str	x = "Hello"	Text / characters
Boolean	bool	x = True	True or False only
List	list	x = [1, 2, 3]	Ordered, mutable
Tuple	tuple	x = (1, 2, 3)	Ordered, immutable
Dictionary	dict	x = {'a': 1}	Key-value pairs
Set	set	x = {1, 2, 3}	Unique unordered items
NoneType	None	x = None	Represents no value

Checking Data Types with type()

```
x = 42

y = 3.14

z = "Python"

b = True


print(type(x)) # <class 'int'>

print(type(y)) # <class 'float'>

print(type(z)) # <class 'str'>

print(type(b)) # <class 'bool'>
```

8. Type Conversion (Type Casting)

You can **convert** one data type to another using built-in conversion functions:

Type Conversion Examples

```
# int() — converts to integer

x = int(3.9) # x = 3 (truncates decimal)
```

```
x = int("25") # x = 25

# float() — converts to float
y = float(10) # y = 10.0
y = float("3.5") # y = 3.5

# str() — converts to string
z = str(100) # z = '100'
z = str(3.14) # z = '3.14'

# bool() — converts to boolean
b = bool(0) # b = False
b = bool(1) # b = True
b = bool("") # b = False (empty string)
```

■ int('3.9') will raise a ValueError. Convert float first: int(float('3.9'))

9. Python Operators

Operators are symbols that perform operations on variables and values.

A) Arithmetic Operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 2	2.5
//	Floor Division	5 // 2	2
%	Modulus	5 % 2	1
**	Exponent	2 ** 3	8

B) Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater or equal	5 >= 5	True
<=	Less or equal	3 <= 5	True

C) Logical Operators

Operator	Description	Example	Result
and	True if both are True	True and False	False
or	True if at least one is True	True or False	True
not	Reverses the boolean value	not True	False

D) Assignment Operators

Operator	Example	Equivalent To
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 2	x = x / 2
**=	x **= 2	x = x ** 2
%=	x %= 3	x = x % 3

10. Input and Output

Python uses **print()** for output and **input()** for reading user input.

The print() Function:

■ print() Examples

```
print("Hello, World!") # Simple output

print("Name:", "Ali") # Multiple values

print(10 + 5) # Prints 15
```

```
print("Line 1\nLine 2") # \n = new line

print("A", "B", "C", sep="-") # Prints A-B-C

print("Hello", end=" ") # No newline at end
```

The input() Function:

The **input()** function always returns a **string**. Convert it if you need a number.

input() Examples

```
name = input("Enter your name: ")

print("Hello,", name)


# Getting a number from user

age = int(input("Enter your age: "))

print("Next year you'll be:", age + 1)


# Getting a float

price = float(input("Enter price: "))
```

f-Strings (Formatted Strings) — Python 3.6+:

f-Strings

```
name = "Sara"

marks = 95


# f-string syntax: f"...{variable}..."

print(f"Student: {name}, Marks: {marks}")

# Output: Student: Sara, Marks: 95


# Expressions inside f-strings

print(f"Next year marks prediction: {marks * 1.05:.1f}")

# Output: Next year marks prediction: 99.8
```

11. Comments in Python

Comments are lines that Python **ignores during execution**. They are used to explain code for human readers.

■ Comment Styles

```
# This is a single-line comment

x = 10 # This comment is at the end of a line

"""
This is a multi-line comment (docstring).
It can span multiple lines.
Often used for function/class documentation.
"""

'''
This also works as a multi-line comment.
'''
```

■ Write comments to explain WHY, not WHAT. Good code should explain itself; comments explain intent.

12. Python Keywords (Reserved Words)

Python has **35 reserved keywords** that cannot be used as variable names. These have special meanings in the language.

False	None	True	and	as	assert	async
await	break	class	continue	def	del	elif
else	except	finally	for	from	global	if
import	in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with	yield

13. Indentation — Python's Superpower

Unlike other languages that use **{ }** braces, Python uses **indentation** (spaces or tabs) to define blocks of code. This is **mandatory** — incorrect indentation causes an **IndentationError**.

■ Indentation Example

```
# CORRECT — 4 spaces indent
if True:
print("This is indented") # inside the if block
print("Still inside")
print("This is outside") # back to main level

# WRONG — will cause IndentationError
if True:
print("No indent!") # ERROR!
```

■ Always use 4 spaces (not Tab) for indentation. Mixing spaces and tabs causes errors in Python 3.

14. Practice Problems with Solutions

Problem 1: Simple Calculator

■ Problem 1: Simple Calculator

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
print(f"Sum: {a + b}")
print(f"Difference: {a - b}")
print(f"Product: {a * b}")
print(f"Quotient: {a / b:.2f}")
```

Problem 2: Swap Two Variables

■ Problem 2: Swap Two Variables

```
x = 10
y = 20
# Method 1: Using a temp variable
temp = x
```

```
x = y

y = temp

print(f"x = {x}, y = {y}")


# Method 2: Python shortcut

x, y = y, x
```

Problem 3: Area of a Circle

Problem 3: Area of a Circle

```
import math

radius = float(input("Enter radius: "))

area = math.pi * radius ** 2

perimeter = 2 * math.pi * radius

print(f"Area: {area:.2f}")

print(f"Perimeter: {perimeter:.2f}")
```

15. Chapter Summary

Topic	Key Point
What is Python?	High-level, interpreted, dynamically typed language
Creator	Guido van Rossum (1991)
print()	Displays output to the screen
input()	Reads user input (always returns string)
Variables	Named containers; no type declaration needed
Data Types	int, float, str, bool, list, tuple, dict, set, None
Type Conversion	int(), float(), str(), bool()
Arithmetic Ops	+ - * / // % **
Comparison Ops	== != > < >= <= (return True/False)
Logical Ops	and, or, not
Comments	# single line """ multi-line """

f-Strings	f"Hello {name}" — embed variables in strings
Indentation	4 spaces — mandatory, defines code blocks
Keywords	35 reserved words cannot be used as variable names

PythonAd